

Cindy Hsu

Garden Grove, CA | twncindy@yahoo.com | Portfolio | LinkedIn | GitHub

PROFESSIONAL SUMMARY

GenAI / LLM application engineer with hands-on, production-grade experience building retrieval-augmented generation (RAG) systems, from hand-rolled primitives through a fully migrated, deployed, LangGraph-orchestrated, stateful multi-turn architecture live on Google Cloud Run. Designed a two-stage retrieval pipeline (ChromaDB + HuggingFace cross-encoder reranking), a provider-agnostic generation layer supporting OpenAI and Gemini with tiered failover, and a Corrective RAG (CRAG) fallback that grounds answers in live web search when local context and trained knowledge both fall short. Brings 12+ years of production software engineering discipline — version control, testing, deployment, incident debugging — to a deliberate, focused transition into applied GenAI/LLM engineering.

Career note: 2019–2025, extended personal sabbatical (travel); 2025–present, self-directed transition into applied AI/ML engineering.

CORE SKILLS

Python, FastAPI, LangGraph, LangChain (langchain-openai, langchain-google-genai), RAG pipeline design, ChromaDB, FAISS (early-stage prototyping), HuggingFace (sentence-transformers, cross-encoder reranking), OpenAI API, Gemini API, multi-provider LLM integration & failover, retry logic & exponential backoff, async/await, prompt engineering, Docker, Git & GitHub, REST APIs, pytest, cloud deployment (Render, Google Cloud Run), environment variable & secrets management, AI-assisted development (Cursor), JSON, command line.

PROJECTS

2026 – Present

AI Chatbot with RAG — LangGraph Edition — Python, FastAPI, LangGraph, LangChain, ChromaDB, HuggingFace, OpenAI API, Gemini API, Docker, Cursor

Code: github.com/cindyhsugit/ai-chatbot-fastapi-with-rag

- Fully migrated orchestration to **LangGraph** and deployed live on **Google Cloud Run**; the original manual (no-framework) implementation remains live on Render as a deliberate before/after architectural comparison.
- Designed a two-stage retrieval architecture: wide-recall vector retrieval (top 20 candidates) followed by HuggingFace cross-encoder reranking (ms-marco-MiniLM-L-6-v2) narrowed to the top 3, improving answer relevance beyond raw vector similarity alone.
- Modeled the pipeline as a typed StateGraph with conditional routing between context-grounded, trained-knowledge, and web-search generation paths, and added a MemorySaver checkpoint with a message-history reducer to give the system persistent multi-turn conversational memory across separate API calls.
- Migrated vector storage from a self-managed FAISS index to ChromaDB and embeddings from OpenAI's paid API to a self-hosted HuggingFace model (all-MiniLM-L6-v2), removing per-call cost and external dependency; diagnosed and resolved a resulting vector-dimensionality mismatch (1536 vs. 384) caused by inconsistent embedding usage across synchronous and asynchronous code paths.
- Built a provider-agnostic generation layer (OpenAI primary, Gemini fallback via LangChain's with_fallbacks) with a separate knowledge-coverage failover that triggers live web search (Tavily API) when neither retrieved context nor trained knowledge can answer a query, grounding the regenerated answer and flagging it as web-sourced rather than presenting it as verified local knowledge.
- Diagnosed a cross-provider response-format bug: Gemini returns structured content blocks where OpenAI returns a plain string, silently breaking the UI; wrote a normalization layer so downstream code stays provider-agnostic.
- Root-caused an intermittent multi-turn memory failure to LLM sampling variance at a decision boundary, not a state bug — ruling out checkpoint, thread-id, and prompt-template causes via repeated trials before fixing it with temperature tuning.
- Verified retrieval correctness through self-directed adversarial testing — injecting fabricated facts into source documents to confirm answers came from retrieved context, not model pretraining — and confirmed correct refusal behavior when a query fell entirely outside the document's scope.
- Used **Cursor's AI coding agent** to close final test-coverage gaps, critically reviewing and validating generated tests rather than accepting them outright; identified and corrected a subprocess-level test-coverage limitation (coverage.py does not track child-process execution), resolving it with a standard coverage-exclusion pattern rather than over-engineering the test.
- Maintained a pytest suite with 100% line coverage (763 statements, 70 tests) including mocked provider calls, graph-node unit tests, and end-to-end integration tests; version-controlled across multiple feature branches, including resolving a diverged branch and merge conflict during a history cleanup.

PRIOR PROFESSIONAL EXPERIENCE — SOFTWARE ENGINEERING

Senior iOS Developer, StubHub, San Francisco March 2017 – May 2019

- Developed new features for different iOS versions, share extensions, widgets, and public/private libraries.
- Enhanced UI behavior for video/audio call states and responsive settings across iPad and iPhone.
- Improved localization support, increasing non-English speaking users by 300%.

Senior iOS Developer, Udemy, San Francisco April 2015 – February 2017

- Handled build and release processes while collaborating with QA overseas.
- Translated the codebase to Swift and increased Swift usage across the project.

Senior iOS Developer, Mercari, San Francisco September 2014 – April 2015

- Worked with the Japan team to maintain code consistency and support new feature development.
- Tested U.S. and Japan app versions and supported localization testing in English and Japanese.

EDUCATION

M.S. Computer Science — University of Southern California, 2005 – 2007

B.S. Computer Science — Iowa State University, 2001 – 2005